

1. Fundamentals

1.1 MOS Transistors

Def. 1. n-type: Closed (ON) when Gate is High (1). Excellent at passing 0 (strong 0).

Def. 2. p-type: Closed (ON) when Gate is Low (0). Excellent at passing 1 (strong 1).

Def. 3. CMOS: Organized into two networks to prevent static power draw:

- **Pull-Up (PUN):** Built with pMOS to connect output to V_{DD} (1).
- **Pull-Down (PDN):** Built with nMOS to connect output to Ground (0).

1.2 Logic Gates

Def. 4. NOT: 2 transistors (1 pMOS, 1 nMOS).

$$Y = \overline{A}$$

NAND: 4 transistors.

$$Y = \overline{A \cdot B}$$

NOR: 4 transistors.

$$Y = \overline{A + B}$$

XOR: (1 if different)

$$Y = A \oplus B$$

XNOR: (1 if same)

$$Y = \overline{A \oplus B}$$

Buffer: (two NOT gates)

$$Y = A$$

AND/OR are usually implemented as NAND/NOR followed by an inverter.

Imp. 5. A set of gates is **logically complete** if any Boolean function can be realized using only those gates (e.g. $\{NAND\}$, $\{NOR\}$).

1.3 Boolean Algebra

Thm. 6. Basic Rules: Identity ($A \cdot 1 = A, A + 0 = A$), Complement ($A \cdot \overline{A} = 0, A + \overline{A} = 1$), Distributive ($A \cdot (B + C) = A \cdot B + A \cdot C$).

Thm. 7. DeMorgan’s theorems:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$
$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Def. 8. Minterm: AND term containing all input variables. **Sum of Products (SOP):** OR of minterms for which output is 1.

Algorithm 9

```
1 #Karnaugh Map (K-Map) format
2 1. Map truth table into grid where adjacent cells
   differ by 1 bit (Gray code).
3 2. Circle groups of 2^n adjacent 1s.
4 3. Eliminate variables that change within the
   circle.
```

2. Combinational Logic

Def. 10. Combinational Logic output is a pure function of current inputs (no memory).

2.1 Building Blocks

Def. 11. An n -to- 2^n **decoder** sets exactly one of its 2^n outputs to 1 based on its n -bit input.

Def. 12. A **Multiplexer (MUX)** uses $\log_2 N$ select lines to route one of N data inputs to the output.

Ex. 13. A MUX can implement any logic function by acting as a **Lookup Table (LUT)**, using variables as select signals.

Def. 14. A **Full Adder:**

$$\text{Sum: } S = A \oplus B \oplus C_{in}$$
$$\text{Carry: } C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$$

Imp. 15. An n -bit **Ripple Carry Adder (RCA)** chains n Full Adders. The critical path is the carry "ripple" from LSB to MSB.

Lem. 16. A **Tri-state buffer** has states: 0, 1, and **High-Z** (disconnected, allowing multiple components to share a bus).

Ex. 17. Programmable Logic Array (PLA): Programmable AND-plane and OR-plane representing a Sum-of-Products.

3. Sequential Logic

Def. 18. Sequential logic incorporates storage elements to maintain "state", producing outputs based on both current and past inputs.

3.1 Fundamental Blocks

Def. 19. Basic storage element: two inverters connected in a feedback loop.

Def. 20. Reset-Set (R-S) latch (built from cross-coupled NAND gates): Set ($S = 0$): $Q = 1$, Reset ($R = 0$): $Q = 0$. $S = 0, R = 0$ is forbidden (metastability).

Def. 21. Gated D Latch adds Write Enable (WE). Transparent when $WE = 1$ (output Q follows D), holds value otherwise.

Def. 22. D Flip-Flop uses two D latches (master-slave) to be **edge-triggered**. Captures D value exactly at rising edge of clock ($0 \rightarrow 1$).

3.2 Timing and Finite State Machines

Def. 23. Clock (CLK): Periodic signal coordinating synchronous state transitions. Period must exceed the worst-case combinational delay.

Def. 24. Finite State Machine (FSM) has:

- **Moore FSM:** Outputs depend **only** on current state.
- **Mealy FSM:** Outputs depend on current state **and** current inputs.

Rem. 25. FSM Implementation: 1. State Register (sequential) 2. Next State Logic (combinational) 3. Output Logic (combinational)

Ex. 26. State Encodings: **Binary** (min bits), **One-Hot** (1 bit/state, simple logic), **Output** (state bits = output bits).

Def. 27. Register: Group of D flip-flops with common clock/enable. **Memory:** Storage locations indexed by address. Hardware uses an **Address Decoder** to select a location and a **Multiplexer** to route data to the output.

4. Timing

4.1 Combinational Timing

Def. 28. Propagation Delay (t_{pd}): Maximum time from input change until output reaches final value. **Contamination Delay (t_{cd}):** Minimum time from input change until output starts to change.

Ex. 29. Glitch: Single input transition causing multiple output transitions due to different path delays. Fix with redundant "consensus terms" in K-maps.

4.2 Sequential Timing

Def. 30. Aperture time ($t_a = t_{setup} + t_{hold}$):

- **Setup Time (t_{setup}):** Time before clock edge data must be stable.
- **Hold Time (t_{hold}):** Time after clock edge data must remain stable.

Violations lead to **metastable** states.

Def. 31. Prop. Clock-to-Q Delay (t_{pcq}): Max time after clock edge until Q is stable. **Cont. Clock-to-Q Delay (t_{ccq}):** Min time after clock edge until Q starts to change.

Thm. 32. For a synchronous system to operate correctly: **Setup Constraint (Max Freq):**

$$T_c \geq t_{pcq} + t_{pd} + t_{setup}$$

Hold Constraint:

$$t_{ccq} + t_{cd} > t_{hold}$$

(Hold time violations cannot be fixed by simply slowing down the clock).

4.3 Clock Skew

Def. 33. Clock skew (t_{skew}): Time difference between arrival of clock signal at different flip-flops. Worsens both constraints:

- **Setup:** $T_c \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew}$
- **Hold:** $t_{ccq} + t_{cd} > t_{hold} + t_{skew}$

5. Von Neumann Model and ISA

Def. 34. The **Von Neumann model** consists of memory (stores program and data), processing unit (ALU, registers), control unit (PC, IR), input, and output. Key properties: **Stored Program** and **Sequential Processing**.

Rem. 35. Instruction Cycle: Fetch → Decode → Evaluate Address → Fetch Operands → Execute → Store Result.

Def. 36. Address Space: Total unique memory locations. **Addressability:** Bits stored per address. **Endianness:** Order of bytes (Big Endian stores MSB at lowest address). To access memory, the **Memory Address Register (MAR)** holds the target address, and the **Memory Data Register (MDR)** receives/holds the data.

5.1 Instruction Set Architecture (ISA)

Def. 37. The **ISA** is the interface between software and hardware. Defines memory organization, register set, and instructions (Opcode + Operands).

Lem. 38. CISC: Complex instructions, small semantic gap. **RISC:** Simple instructions, large semantic gap (e.g. MIPS, LC-3).

Def. 39. Addressing Modes: Immediate, Register, PC-Relative ($PC + offset$), Indirect, Base + Offset.

5.2 Assembly

Def. 40. Assembly is human-readable machine code. In RISC, operands must be in registers for ALUs:

- **Load:** Reads data from memory into a register.
- **Store:** Writes data from a register into memory.

Rem. 41. Execution order altered by **Conditional Branches** (uses condition codes like N, Z, P or direct register comparisons) and **Unconditional Jumps**.

6. Microarchitecture

Def. 42. Microarchitecture implements an ISA. One ISA can have many microarchitectures (single-cycle, multi-cycle, pipelined) optimized for **Performance**, **Cost** (die area), **Power/Energy**, and **Reliability**.

6.1 Single-Cycle Microarchitecture

Def. 43. Every instruction takes exactly one clock cycle.

- **CPI:** Always 1.
- **Clock Cycle Time (T_c):** Bounded by the slowest instruction (usually 1w).

6.2 Multi-Cycle Microarchitecture

Def. 44. Breaks instruction execution into multiple shorter cycles. Cycles take variable steps. Allows **hardware reuse** (e.g. ALU) and requires internal microarchitectural registers (IR, MDR, ALUOut).

Ex. 45. Phases: 1. Fetch, 2. Decode, 3. Execute, 4. Memory, 5. Write-back.

Thm. 46.

$$Time = \#instr \times Average_CPI \times T_c$$

Multi-cycle has higher CPI but shorter T_c , improving performance over single-cycle. Downside: **Sequencing overhead** added at each cycle transition.

Rem. 47. Multi-cycle control unit is implemented as an FSM to track instruction steps.

7. Pipelining

Def. 48. Pipelining overlaps multiple instructions in execution (assembly line logic). Increases **instruction throughput** but does not decrease latency.

Thm. 49. A k -stage pipeline ideal speedup is k -fold, but limited by unbalanced stages, sequencing overhead, and hazards.

7.1 The 5-Stage Pipeline

Def. 50. Stages (MIPS): 1. Fetch (IF), 2. Decode (ID), 3. Execute (EX), 4. Memory (MEM), 5. Write-Back (WB). **Pipeline Registers** (e.g. IF/ID) pass data and control signals forward between stages.

7.2 Hazards

Def. 51. A hazard prevents the next instruction from executing normally:

- **Structural:** Conflict for hardware resources.
- **Data:** Instruction depends on data from a non-completed instruction.
- **Control:** Branch outcome delays fetching correct next instruction.

Def. 52. Data dependencies: **Read-After-Write (RAW):** Instruction needs data about to be written. (Only hazard needing hardware fix in 5-stage pipeline).

7.3 Resolving Hazards

Lem. 53. Forwarding (Bypassing) routes result from EX or MEM directly to ALU input.

Imp. 54. Load-Use Hazard cannot be forwarded. Requires a **Stall** (freeze PC, insert pipeline bubble).

Rem. 55. Control Hazards force a pipeline flush on misprediction. Strategies: Static prediction, Dynamic prediction (Branch History Table), Early Resolution (moving compare to Decode).

7.4 Advanced Issues

Def. 56. Precise Exceptions: Must stop execution on exception instruction while ensuring prior instructions finish correctly. Unplanned internal changes are **exceptions**, external are **interrupts**. Hardware must flush younger instructions and save PC/REG.

Def. 57. Out-of-Order Completion: Multi-cycle units cause instructions to finish out of order, complicating exceptions. **Reorder-Buffer (ROB):** Stores results of instructions as they complete. Allows out-of-order completion but **in-order retirement** to maintain precise exceptions. Uses a valid bit in REG to point to ROB if not ready.

Def. 58. Fine-Grained Multithreading: Switches thread every clock to hide hazards.